

Bitwise

Counting Bits

```
#include

using namespace std;
typedef long long ll;

ll N;
map dp;

ll f(ll a){
    if(a == 0) return 0;
    if(dp[a]) return dp[a];
    if(a&1) return dp[a] = 2*f(a/2)+(a/2)+1;
    else return dp[a] = f(a/2)+f(a/2-1)+(a/2);
}

int main(){
    scanf("%lld", &N);
    printf("%lld\n", f(N));
}
```

Maximum Xor Subarray

```
#include

using namespace std;

struct Node {
    Node *c[2];
};

int N, xum, best;
Node *root;

void update(int x){
    Node *cur = root;
    for(int i = 30; i >= 0; i--){
        if(x&(1<c[1]) cur->c[1] = new Node();
        cur = cur->c[1];
    } else {
        if(!cur->c[0]) cur->c[0] = new Node();
        cur = cur->c[0];
    }
}

int query(int x){
    int res = 0;
    Node *cur = root;
    for(int i = 30; i >= 0; i--){
        if(x&(1<c[0])){
```

```

        res += (1<c[0];
    } else cur = cur->c[1];
} else {
    if(cur->c[1]){
        res += (1<c[1];
    } else cur = cur->c[0];
}
}
return res;
}

int main(){
    scanf("%d", &N);
    root = new Node();
    update(0);
    for(int i = 0, x; i < N; i++){
        scanf("%d", &x);
        xum ^= x;
        update(xum);
        best = max(best, query(xum));
    }
    printf("%d\n", best);
}

```

Maximum Xor Subset

```

#include

using namespace std;
const int logX = 32;

int N, basis[logX];

void insert_vector(int mask){
    for(int i = logX-1; i >= 0; i--){
        if((mask&(1<= 0; i--){
            if(!basis[i]) continue;
            if((ans&(1<

```

Number of Subset Xors

```

#include

using namespace std;
typedef long long ll;
const int logX = 32;

int N, basis[logX];

bool insert_vector(int mask){

```

```
for(int i = logX-1; i >= 0; i--){
    if((mask&(1<
```

Xor Pyramid Peak

```
#include

using namespace std;
const int maxN = 2e5+5;

int N, D, a, xum;

int two_divisibility(int x){
    int res = 0;
    while(x){
        x >>= 1;
        res += x;
    }
    return res;
}

int main(){
    scanf("%d", &N);
    D = two_divisibility(N-1);
    for(int i = 0; i < N; i++){
        scanf("%d", &a);
        int d1 = two_divisibility(i);
        int d2 = two_divisibility(N-i-1);
        if(D - d1 - d2 == 0)
            xum ^= a;
    }
    printf("%d\n", xum);
}
```

Xor Pyramid Diagonal

```
#include

using namespace std;
const int maxN = 2e5;

int N, a[maxN];

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++)
        scanf("%d", &a[i]);

    for(int i = 0; i < N; i++){
        int ans = a[0];
        for(int subset = i; subset > 0; subset = (subset-1)&i)
            ans ^= a[subset];
    }
}
```

```
        printf("%d%c", ans, (" \n")[i==N-1]);  
    }  
}
```
